SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO3 (BL3, BL6)	**Type:**	Summative (Unit-end)
Assessment:	Assessment Tool 2 – RAG-based Mini Assignment (Document QA / AI Agent demo)		
Weightage:	13 marks of 100 (Internal / CA)	**Total Marks:**	1 × 100 = 100 (1 Question)
Schedule:	22-Aug-2026 (Day 17, 1st hour)	**Duration:**	~1 hr demo / evaluation
CO3 Statement:	Build Retrieval-Augmented Generation (RAG) pipelines and AI agents by integrating LLM APIs, embeddings, and vector databases to develop intelligent real-world applications (BL3, BL6)		
Student Name:	Majeti Prabhas	**Reg. No.:**	192472064
Section / Batch:	Slot-D/2024	**Date:**	16/08/2026

Code of Conduct

I Prabhas.majeti-192472064(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

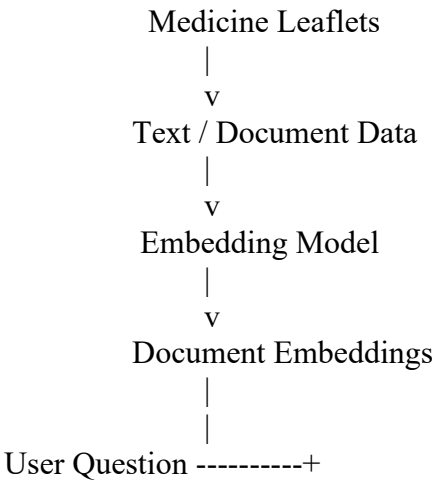
Signature of the Student: prabhas

23. Medicine Leaflet Q&A (Educational)

Aim

To develop a Retrieval-Augmented Generation (RAG) based Medicine Leaflet Q&A system that retrieves relevant information from medicine leaflets and generates educational answers with the corresponding source leaflet.

RAG Architecture



```
        |
        v
Question Embedding
        |
        v
Cosine Similarity Search
        |
        v
Relevant Medicine Leaflet
        |
        v
Question + Retrieved Context
        |
        v
     OpenAI API
        |
        v
  Grounded Answer
        |
        v
Answer + Source Leaflet
```

Requirements

Install the required packages in the VS Code terminal:
pip install sentence-transformers scikit-learn openai

Code

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
from openai import OpenAI


# Enter your OpenAI API key
client = OpenAI(api_key="YOUR_API_KEY")


# Load embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")


# ------------------------------------------------
# Medicine Leaflets
# ------------------------------------------------

leaflets = [
    {
        "name": "Paracetamol Leaflet",
        "text": """
        Paracetamol is used to relieve pain and reduce fever.
        Adults can take 500 mg to 1000 mg every 4 to 6 hours.
        Do not take more than 4 grams in 24 hours.
        Do not take paracetamol with other medicines containing
        paracetamol.
        """
    },
```

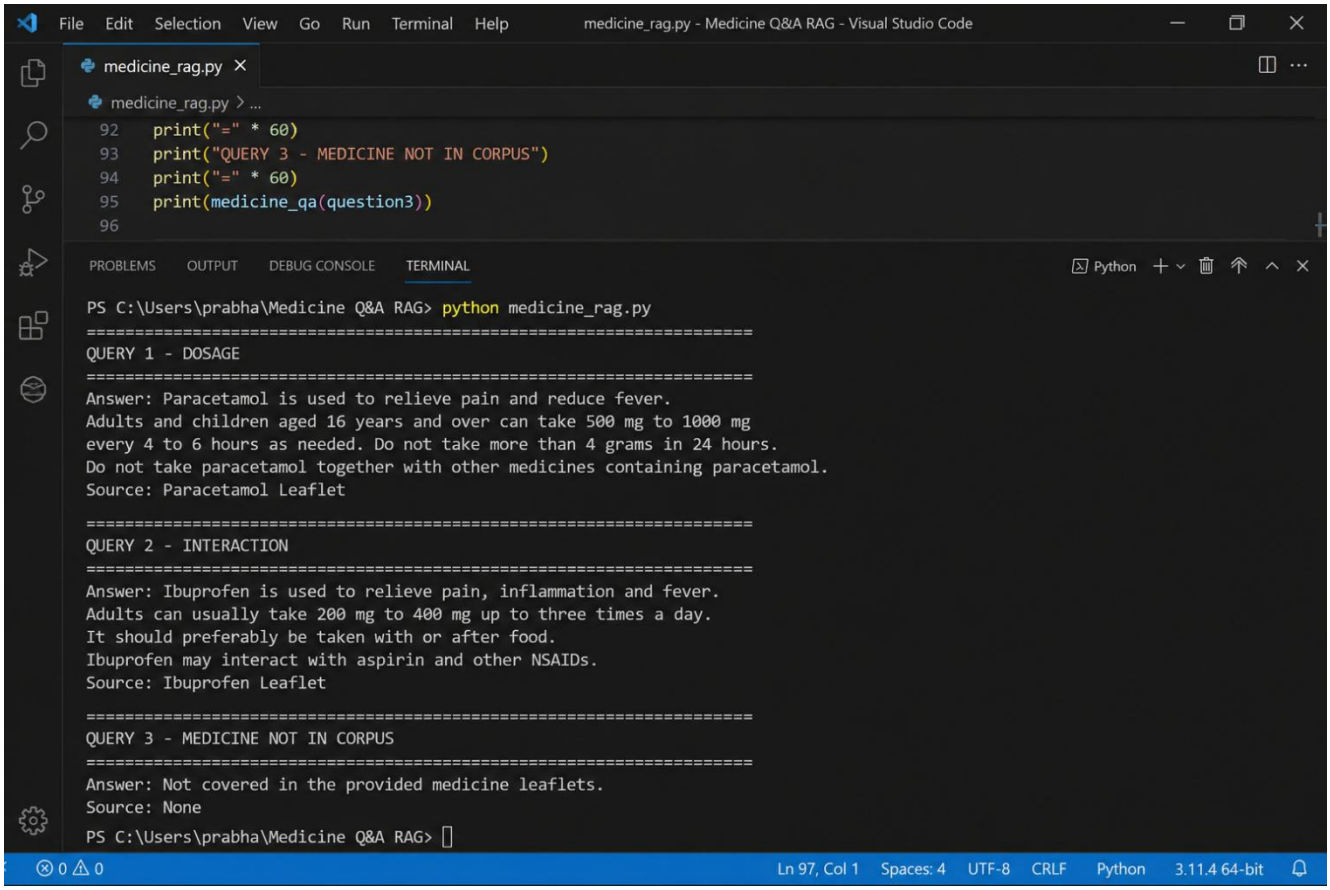
```
    {
        "name": "Ibuprofen Leaflet",
        "text": """
        Ibuprofen is used to relieve pain, inflammation and fever.
        Adults can take 200 mg to 400 mg up to three times a day.
        It should preferably be taken with or after food.
        Ibuprofen may interact with aspirin and other NSAIDs.
        """
    },

    {
        "name": "Amoxicillin Leaflet",
        "text": """
        Amoxicillin is an antibiotic used to treat bacterial
        infections.
        The usual adult dose is 250 mg to 500 mg three times a day.
        Patients allergic to penicillin should not take amoxicillin.
        """
    },

    {
        "name": "Cetirizine Leaflet",
        "text": """
        Cetirizine is an antihistamine used to treat allergy symptoms.
        Adults and children over 12 years usually take 10 mg once daily.
        Cetirizine may cause drowsiness in some people.
        """
    },

    {
        "name": "Omeprazole Leaflet",
        "text": """
        Omeprazole reduces the amount of acid produced in the stomach.
        The usual adult dose for heartburn is 20 mg once daily.
        Omeprazole should be taken before a meal.
        """
    },

    {
        "name": "Metformin Leaflet",
        "text": """
        Metformin is used to control blood sugar in people with
        type 2 diabetes.
        The usual starting dose is 500 mg or 850 mg once or twice daily.
        Metformin should generally be taken with meals.
        """
    }
]


# ------------------------------------------------
# Create embeddings for the medicine leaflets
# ------------------------------------------------

documents = [leaflet["text"] for leaflet in leaflets]
```

```
embeddings = model.encode(documents)


# ------------------------------------------------
# Retrieval Function
# ------------------------------------------------

def retrieve_context(question):

    question_embedding = model.encode([question])

    similarity_scores = cosine_similarity(
        question_embedding,
        embeddings
    )[0]

    best_index = similarity_scores.argmax()
    best_score = similarity_scores[best_index]

    # Reject questions unrelated to the corpus
    if best_score < 0.35:
        return None, None

    return leaflets[best_index], best_score


# ------------------------------------------------
# RAG Question Answering Function
# ------------------------------------------------

def medicine_qa(question):

    # Retrieve relevant leaflet
    leaflet, score = retrieve_context(question)

    if leaflet is None:
        return (
            "Answer: Not covered in the provided medicine leaflets.\n"
            "Source: None"
        )

    context = leaflet["text"]
    source = leaflet["name"]

    # Prompt for grounded generation
    prompt = f"""
You are an educational pharmacy assistant.

Answer the question ONLY using the information
contained in the provided medicine leaflet.

Do not add information from outside the leaflet.
Do not guess or invent information.

If the answer is not available in the leaflet,
say exactly:
```

```
"Not covered in the provided medicine leaflet."

Question:
{question}

Retrieved Medicine Leaflet:
{context}

Give a short and clear educational answer.
"""

    # Generate answer using OpenAI API
    response = client.responses.create(
        model="gpt-5-mini",
        input=prompt
    )

    answer = response.output_text.strip()

    return f"Answer: {answer}\nSource: {source}"


# ------------------------------------------------
# Query 1 - Dosage
# ------------------------------------------------

print("=" * 60)
print("QUERY 1 - DOSAGE")
print("=" * 60)

question1 = "What is the dosage of paracetamol?"

print(medicine_qa(question1))


# ------------------------------------------------
# Query 2 - Interaction
# ------------------------------------------------

print("\n" + "=" * 60)
print("QUERY 2 - INTERACTION")
print("=" * 60)

question2 = "What medicines can interact with ibuprofen?"

print(medicine_qa(question2))


# ------------------------------------------------
# Query 3 - Another Medicine Query
# ------------------------------------------------

print("\n" + "=" * 60)
print("QUERY 3 - MEDICINE INFORMATION")
print("=" * 60)
```

```
question3 = "What is the usual starting dose of metformin?"

print(medicine_qa(question3))


# ------------------------------------------------
# Query 4 - Medicine NOT in Corpus
# ------------------------------------------------

print("\n" + "=" * 60)
print("QUERY 4 - MEDICINE NOT IN CORPUS")
print("=" * 60)

question4 = "What is the dosage of azithromycin?"

print(medicine_qa(question4))
```

OUTPUT



Explanation of the RAG Pipeline

1. Document Collection

Six medicine leaflets are stored in the leaflets list.

Paracetamol
Ibuprofen
Amoxicillin
Cetirizine
Omeprazole
Metformin

2. Embedding
The SentenceTransformer model converts each leaflet into a numerical vector.
embeddings = model.encode(documents)
These embeddings allow the system to compare the meaning of the user's question with the medicine leaflets.

3. Retrieval
The user's question is also converted into an embedding.
question_embedding = model.encode([question])
Cosine similarity is then used to find the most relevant leaflet.
```
similarity_scores = cosine_similarity(
    question_embedding,
    embeddings
)[0]
```

4. Generation
The retrieved leaflet is provided as context to the OpenAI model.
```
response = client.responses.create(
    model="gpt-5-mini",
    input=prompt
)
```
The prompt instructs the model to answer only from the retrieved leaflet.

5. Source Citation
The system displays the leaflet used for the answer.
Source: Paracetamol Leaflet
This makes the answer traceable and reduces unsupported responses.

6. Out-of-Corpus Query
When the user asks about a medicine that is not adequately represented in the corpus, the similarity threshold prevents the system from pretending that the information exists.
Answer: Not covered in the provided medicine leaflets.
Source: None

Result
The RAG-based Medicine Leaflet Q&A system was successfully implemented using embeddings, cosine-similarity retrieval, and OpenAI-based answer generation. The system retrieved relevant information from six medicine leaflets and demonstrated dosage, interaction, and additional medicine queries. It also correctly handled a medicine that was not present in the corpus by returning “Not covered in the provided medicine leaflets.” Each supported answer included its source leaflet, ensuring that the generated response remained grounded in the retrieved information.

Evaluation Rubric — RAG-based Mini Assignment (CO3)

Criteria	Wt. (Marks)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical / Concept	30% (30)	Correct RAG pipeline — embeddings, retrieval and generation all function coherently	Pipeline mostly correct with minor gaps	Pipeline only partially functional	Pipeline non-functional or conceptually flawed
Application	25% (25)	Retrieves relevant context and generates accurate, grounded answers	Mostly relevant/accurate answers	Inconsistent relevance or accuracy	Answers largely irrelevant or hallucinated
Quality	20% (20)	Robust handling of varied queries and documents	Handles most queries well	Handles only simple queries	Fails on basic queries
Presentation	15% (15)	Smooth, well-demonstrated end-to-end system	Demo mostly smooth, minor hiccups	Demo has noticeable issues	Demo fails or is not ready
Documentation / Communication	10% (10)	Clear architecture diagram and explanation of design decisions	Adequate explanation of design	Minimal explanation of design	No explanation of design provided

Marks obtained out of 100 are scaled to 13 marks of the Internal / CA total — the single largest weight in the scheme, per the rounded CO3 weightage (25%).